تمرین اول: ساختار کلاس‌ها و تحلیل اصول ۵گانه SOLID

بررسی ساختار کلاس‌ها و نحوه تقسیم مسئولیت‌ها در پروژه «ژتون» (Zheton)

تهیه و تدوین: پویا رئیس‌نسب (Pouya RaisiNasab) | درس آزمایشگاه مهندسی نرم‌افزار | مخزن رسمی گیت‌هاب: github.com/PouyaRN/Zheton

۱. مشخصات نرم‌افزار فعلی ژتون و تاریخچه تغییرات و تکامل معماری

این تمرین بر اساس آخرین نسخه فعال نرم‌افزار «**ژتون**» (**Zheton**) پیاده‌سازی شده است. ژتون یک دستیار هوشمند مالی، دیده‌بان قدرت خرید و بودجه‌بندی اول‌محلی (Local-First) برای اندروید است که بر روی گیت‌هاب توسعه داده می‌شود. مشخصات فنی و تغییرات اعمال‌شده نسبت به نسخه‌های اولیه به شرح زیر است:

مشخصه سیستم	شرح مشخصات و تغییرات نرم‌افزار در کدبیس فعال ژتون
نام و مخزن گیت‌هاب	**ژتون (Zheton)** – آدرس رسمی ریپازیتوری: github.com/PouyaRN/Zheton (منشعب از هسته Andis)
تغییرات ساختاری و معماری	ادغام صفحات پراکنده در ۴ تب منسجم طبق ADR 0007 (داشبورد، بازار و طلا، حساب‌ها و تعهدات، دفترکل و تحلیل ۵۰/۳۰/۲۰)؛ پیاده‌سازی سیستم طراحی اختصاصی Lithe Vitality بدون هرگونه داده ماک.
موتورهای محاسباتی جدید	افزودن موتور خرج امن روزانه (`SafeSpendEngine`) با اوزان پایان هفته ایران (پنج‌شنبه ۱٫۴۰ و جمعه ۱٫۲۵)، دیده‌بان افت ارزش تورمی ریال راکد، و رادار حباب سکه و صندوق‌های طلا.
تکامل لایه داده و دیتابیس	تفکیک پایگاه‌داده به دو فایل مجزا: `durable.db` (ذخیره قطعی پیامک‌ها و قواعد کاربر) و `derived.db` (محاسبات تجدیدپذیر دفترکل جهت امکان ارتقای پارسر بدون ریسک پاک شدن داده‌ها).
پشته فنی و بیلد پروژه	کاتلین (Kotlin 2.0)، رابط کاربری اعلانی Jetpack Compose، پایگاه‌داده Room 2.7، و انتشار خودکار APK با پایپ‌لاین GitHub Actions.

۲. ساختار کلاس‌های اصلی سیستم (پاسخ سوال‌های ۱، ۲ و ۳)

از میان کلاس‌های پیاده‌سازی‌شده در هسته نرم‌افزار ژتون، ۵ کلاس بنیادین زیر تحلیل شده‌اند (کلاس‌های ۱ و ۲ در این صفحه و کلاس‌های ۳ تا ۵ در صفحه بعد):

نام کلاس و نقش	مسئولیت در معماری (SRP)	ویژگی‌های کلیدی (Attributes)	عملیات و متدها (Methods)
BankAccount (Domain Entity)	نگهداری و کپسوله‌سازی اطلاعات کارت بانکی، شماره شبا، تراز مانده، اعتبارسنجی کارت با الگوریتم Luhn و تشخیص حساب‌های راکد و بدون تراکنش.	• `bank: Bank` (بانک عضو شتاب) • `mask: String` (۴ رقم انتهای کارت) • `balance: Double` (مانده به تومان) • `fullPan: String` (شماره ۱۶ رقمی کارت) • `sheba: String` (شناسه ۲۴ رقمی IR) • `anchored: Boolean` (تأیید با پیامک)	• `isDormant(ms): Boolean` (حساب راکد >۴۵ روز) • `trusted(): Boolean` (تأیید قطعیت مانده) • `validateLuhn(): Boolean` (صحت‌سنجی ساختار کارت) • `bankFa(): String` (نام فارسی رسمی بانک)
BankSms (Value Object)	مدل‌سازی پیامک بانکی، تفکیک جهت واریز/برداشت، مبلغ، مانده جدید و کارمزد با ایمنی نوع داده جهت جلوگیری از پردازش مستقیم متن خام پیامک.	• `bank: Bank` (بانک صادرکننده پیامک) • `sender: String` (سرشماره فرستنده) • `amountRial: Long?` (مبلغ به ریال) • `balanceRial: Long?` (مانده جدید) • `feeRial: Long?` (کارمزد کسرشده) • `delta: Double?` (میزان تغییر با علامت)	• `empty(): Boolean` (فیلتر پیامک غیرمالی) • `isDeposit(): Boolean` (تشخیص واریز یا برداشت) • `toDerivedTxn(): Txn` (تبدیل به ردیف دیتابیس) • `cleanMerchant(): String` (پیرایش نام پذیرنده)

۲. ادامه ساختار کلاس‌های اصلی سیستم (کلاس‌های ۳ تا ۵)

سه کلاس بعدی که تکمیل‌کننده موتور محاسبات اقتصادی، مدیریت تعهدات مالی و هماهنگی تراکنش‌های دیتابیس در ژتون هستند:

نام کلاس و نقش	مسئولیت در معماری (SRP)	ویژگی‌های کلیدی (Attributes)	عملیات و متدها (Methods)
Commitment (Liability Entity)	رهگیری اقساط بانکی، چک صیادی و طرح‌های خرید قسطی (BNPL) و محاسبه فرسایش تورمی ارزش اقساط آینده کاربر.	• `id: String` (شناسه یکتای تعهد مالی) • `title: String` (عنوان بدهی یا قسط) • `kind: CommitmentKind` (وام، چک، BNPL) • `remainingRial: Long` (مانده بدهی) • `installmentCount: Int` (کل اقساط) • `nextDueTimestamp: Long` (سررسید)	• `progressPercent(): Float` (درصد تسویه‌شده) • `discountedPV(m): Long` (تنزیل تورمی قسط) • `inflationRelief(): Long` (ارزش کاهش‌یافته) • `nextDueDays(): Int` (روز مانده تا سررسید)
SafeSpendEngine (Domain Service)	موتور محاسباتی خالص ریاضی برای تعیین خرج امن روزانه، اعمال ضرایب الگوی آخر هفته در ایران و پیش‌بینی روز دقیق اتمام بودجه.	• `WEEKEND_WEIGHT = 1.40` (پنج‌شنبه) • `FRIDAY_WEIGHT = 1.25` (جمعه) • `BASE_WEIGHT = 1.00` (شنبه تا سه‌شنبه) • `MIN_BUFFER_RATIO = 0.10` (ذخیره)	• `calculatePacing(...): Decision` (محاسبه خرج) • `dayOfWeekWeight(d): Double` (ضریب تقویم) • `iranianDayOfWeek(t): Int` (شاخص روز ایران) • `projectExhaustionDay(...): Int?` (روز اتمام)
LedgerEngine (App Orchestrator)	مدیریت و هماهنگی پایگاه‌داده، تضمین قفل‌های همزمانی، بازسازی مجدد دفترکل از روی پیامک‌ها و ثبت قواعد دسته‌بندی خودکار.	• `durable: DurableDb` (دیتابیس ماندگار) • `derived: DerivedDb` (دیتابیس تجدیدپذیر) • `mutex: Mutex` (قفل ایزولاسیون تراکنش‌ها) • `extraBankNumbers: Provider` (انتزاع)	• `withLedgerLock(block): T` (اجرای امن در IO) • `rederive(numbers): Int` (بازسازی کل دفترکل) • `categorise(entry, catId, always)` (قاعده دسته) • `loadSession(): Session` (سشن جاری کاربر)

۳. تحلیل اصول SOLID رعایت‌شده در معماری پروژه ژتون (پاسخ سوال ۴)

در طراحی ساختار کلاس‌ها و معماری داده‌های پروژه، اصول زیر از اصول پنج‌گانه شیءگرایی پیاده‌سازی و رعایت شده‌اند:

اصل شیءگرا (SOLID)	نحوه پیاده‌سازی و شواهد در کدبیس ژتون	دستاورد معماری در پروژه
اصل تک‌مسئولیتی (Single Responsibility - SRP)	کلاس `SafeSpendEngine` منحصراً محاسبات مالی بودجه را انجام می‌دهد و هیچ کدی از UI یا دیتابیس ندارد. همچنین پایگاه‌داده به دو فایل مجزا شکسته شده: `DurableDb` فقط داده‌های قطعی کاربر را نگه می‌دارد و `DerivedDb` فقط محاسبات تجدیدپذیر دفترکل را میزبانی می‌کند.	جداسازی کامل منطق کسب‌وکار از جزئیات ذخیره‌سازی و ارتقای تست‌پذیری
اصل تفکیک رابط‌ها (Interface Segregation - ISP)	در لایه دیتابیس Room، به جای ایجاد یک اینترفیس بزرگ و غول‌پیکر (Fat DAO)، اینترفیس‌ها به بخش‌های مجزا و تک‌منظوره تفکیک شده‌اند: `SmsSourceDao` (پیامک‌ها)، `CategoryDao` (دسته‌ها)، `RuleDao` (قوانین) و `InstallmentDao` (اقساط).	کاهش وابستگی کلاینت‌ها و عدم اجبار به متدهای نامربوط
اصل وارونگی وابستگی (Dependency Inversion - DIP)	کلاس‌های سطح بالا مانند `LedgerEngine` وابستگی‌های خود را مستقیماً از کلاس‌های سخت‌کد شده دریافت نمی‌کنند، بلکه از طریق سازنده (Constructor) و اینترفیس‌های تابعی انتزاعی مثل `extraBankNumbersProvider` تغذیه می‌شوند.	امکان اجرای تست‌های واحد ایزوله بدون نیاز به شبیه‌ساز اندروید

۴. شناسایی نقض اصل SOLID و راهکار بازآرایی (پاسخ سوال ۵)

۴-۱. تحلیل فنی نقض اصل باز/بسته (Open/Closed Principle - OCP)

محل دقیق در پروژه: تابع `parseBankSms` در فایل `Sms.kt` .

شرح مسئله: در کدهای فعلی پروژه، برای پشتیبانی از فرمت‌های اختصاصی پیامک در هر بانک (مثلاً واریزهای گروهی در بانک سپه یا برچسب نوع تراکنش در بانک تجارت)، شرط‌های زنجیره‌ای متوالی به صورت هاردکد درون بدنه تابع اصلی نوشته شده است:

کد فعلی در Sms.kt (نقض صریح OCP):

```
fun parseBankSms(sender: String, body: String): BankSms? { val bank = bankOf(sender) ?: return null var deposit = IN_WORDS.any { body.contains(it) } var withdrawal = OUT_WORDS.any { body.contains(it) } // با افزودن هر بانک جدید، این تابع اصلی دستکاری می‌شود if (bank == Bank.SEPAH && body.contains("واریز گروهی")) { deposit = true; withdrawal = false } if (bank == Bank.TEJARAT) { val type = body.substringAfter("نوع تراکنش:") if (type.contains("واریز")) deposit = true } return BankSms(bank, sender, ...) }
```

کد بازآرایی‌شده با الگوی Strategy (رعایت OCP):

```
// ۱. تعریف اینترفیس استراتژی انتزاعی interface BankSmsStrategy { fun canHandle(bank: Bank): Boolean fun parseCustom(body: String, base: BankSms): BankSms } // ۲. توسعه رفتار بدون دستکاری موتور اصلی class SepahStrategy : BankSmsStrategy { ... } class TejaratStrategy : BankSmsStrategy { ... } // ۳. موتور اصلی نسبت به تغییر بسته و به توسعه باز است class BankSmsParser(private val strategies: List) { fun parse(sender: String, body: String): BankSms? { val bank = bankOf(sender) ?: return null val base = parseCommon(bank, sender, body) val strategy = strategies.firstOrNull { it.canHandle(bank) } return strategy?.parseCustom(body, base) ?: base } }
```

۴-۲. دستاورد معماری بازآرایی (Refactoring)

طبق اصل OCP، نرم‌افزار باید برای «توسعه» باز و برای «تغییر» بسته باشد. در کد اولیه، با اضافه شدن هر بانک جدید به شبکه شتاب کشور، برنامه‌نویس باید فایل اصلی پارسر را باز کرده و بدنه آن را تغییر می‌داد که ریسک ایجاد رگرسیون برای سایر بانک‌ها را بالا می‌برد. با پیاده‌سازی الگوی طراحی استراتژی (Strategy Pattern)، افزودن یک بانک جدید صرفاً با تعریف یک کلاس مستقل انجام شده و کلاس مرکزی هرگز دستکاری نمی‌شود.

۵. خلاصه و دستاوردهای مهندسی سیستم ژتون

معیار ارزیابی	شرح سازوکار پیاده‌سازی و نتیجه در نرم‌افزار ژتون
تفکیک دامنه‌محور (DDD)	مستقل‌سازی کامل هسته محاسبات خرج امن (`SafeSpendEngine`) و موجودیت‌های مالی از فریم‌ورک‌های سیستم‌عامل اندروید.
پایداری تراکنش‌ها	استفاده از قفل‌های همزمانی (Mutex) در `LedgerEngine` جهت حذف امکان تداخل پردازش پیامک‌های پس‌زمینه با ورودی‌های دستی کاربر.
انعطاف‌پذیری و نگهداری‌پذیری	کپسوله‌سازی منطق پیامک بانک‌ها در قالب استراتژی‌های مجزا، تضمین‌کننده مقیاس‌پذیری نرم‌افزار در برابر تغییرات شبکه شتاب است.

نتیجه‌گیری پایانی: در پروژه ژتون (github.com/PouyaRN/Zheton)، طراحی کلاس‌ها با رعایت تعادل میان سادگی و تفکیک وظایف انجام گرفته است. اصول تک‌مسئولیتی، تفکیک رابط‌ها و وارونگی وابستگی در ساختار جاری رعایت شده و با اصلاح تابع پارسر از طریق الگوی استراتژی، اصل باز/بسته نیز به صورت استاندارد در کل سیستم محقق می‌گردد.